

# FIXPOINT SWARM

## Phase 2: Operational Software Layer

### Build Specification for Coding Agent

Evolving the CLI prototype into a live-capable,  
persistent, observable trading mirror with TUI dashboard

- Input:** Completed Phase 1 repository (14 crates, 7 FSMs, CLI, paper-mode, macro-cycle engine)
- Output:** Phase 2 repository with TUI dashboard, persistence, live venue adapters, observability, and TTCP analysis engine
- Constraint:** Do not break Phase 1 functionality.  
All existing tests must continue to pass.
- Date:** 12 March 2026

#### Prime Directive

Phase 2 builds **on top of** Phase 1. No rewriting of core logic. No changing the 14-crate structure. New functionality is added as new modules within existing crates or as at most 2 new crates. Every Phase 1 CLI command must still work identically after Phase 2.

# Contents

---

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>1</b>  | <b>Phase 2 Scope and Non-Scope</b>          | <b>3</b>  |
| 1.1       | What Phase 2 Adds . . . . .                 | 3         |
| 1.2       | What Phase 2 Does NOT Add . . . . .         | 3         |
| 1.3       | New Crates . . . . .                        | 3         |
| <b>2</b>  | <b>TUI Dashboard</b>                        | <b>4</b>  |
| 2.1       | Technology . . . . .                        | 4         |
| 2.2       | Layout . . . . .                            | 4         |
| 2.3       | Panel Specifications . . . . .              | 5         |
| 2.4       | Shared State Architecture . . . . .         | 5         |
| 2.5       | Keybindings . . . . .                       | 5         |
| 2.6       | TUI Launch . . . . .                        | 6         |
| <b>3</b>  | <b>Persistence Layer</b>                    | <b>6</b>  |
| 3.1       | What Gets Persisted . . . . .               | 6         |
| 3.2       | Chain Persistence . . . . .                 | 6         |
| 3.3       | Snapshot and Resume . . . . .               | 7         |
| 3.4       | Compaction . . . . .                        | 7         |
| <b>4</b>  | <b>Live Venue Adapter: Binance Spot</b>     | <b>7</b>  |
| 4.1       | Scope . . . . .                             | 7         |
| 4.2       | Implementation . . . . .                    | 7         |
| 4.3       | WebSocket Order Book Feed . . . . .         | 8         |
| 4.4       | Configuration . . . . .                     | 8         |
| <b>5</b>  | <b>Observability</b>                        | <b>9</b>  |
| 5.1       | Structured Logging . . . . .                | 9         |
| 5.2       | Metrics Export . . . . .                    | 9         |
| <b>6</b>  | <b>TTCP Analysis Engine</b>                 | <b>10</b> |
| 6.1       | Integration Point . . . . .                 | 10        |
| 6.2       | Embedding . . . . .                         | 10        |
| 6.3       | Simplified TTCP for Phase 2 . . . . .       | 10        |
| 6.4       | TTCP Configuration . . . . .                | 11        |
| <b>7</b>  | <b>Configuration Hot-Reload</b>             | <b>11</b> |
| 7.1       | Mechanism . . . . .                         | 11        |
| 7.2       | What Can Be Changed at Runtime . . . . .    | 11        |
| 7.3       | What Cannot Be Changed at Runtime . . . . . | 11        |
| <b>8</b>  | <b>Implementation Order</b>                 | <b>12</b> |
| <b>9</b>  | <b>New Dependencies</b>                     | <b>13</b> |
| <b>10</b> | <b>Updated CLI Surface</b>                  | <b>13</b> |

---

|                                            |           |
|--------------------------------------------|-----------|
| <b>11 File System Layout After Phase 2</b> | <b>13</b> |
| <b>12 Definition of Done (Phase 2)</b>     | <b>14</b> |
| <b>13 Phase 3 Preview (Non-Normative)</b>  | <b>15</b> |

# 1 Phase 2 Scope and Non-Scope

---

## 1.1 What Phase 2 Adds

1. **TUI Dashboard** (Terminal User Interface via `ratatui`): real-time visualization of macro-cycle state, FSM states, resonance metrics, gate scores, P&L, and event log. Solo-operator cockpit.
2. **Persistence Layer**: save/load system state to disk. Resume from checkpoint. Rotate and compact shadow-chain artifacts on disk.
3. **Live Venue Adapter**: at least one real exchange connector (Binance Spot REST+WebSocket) behind the `-features live` gate. The system can observe real order books in paper-mode (no trading) or execute in live-mode.
4. **Observability**: structured logging via `tracing`, metrics export (Prometheus-compatible), health endpoint.
5. **TTCP Analysis Engine**: the Temporal Triangulation Cascade Protocol (v2.0.0) integrated as an analysis subsystem that runs alongside the macro-cycle, triangulating the resonance metric stream and emitting Crystals when topological stability is detected.
6. **Configuration Hot-Reload**: change YAML config without restarting. Validated before swap. Evidence-logged.

## 1.2 What Phase 2 Does NOT Add

- No web GUI, no REST API, no browser dashboard.
- No additional exchanges beyond Binance (others are Phase 3).
- No ML/AI components.
- No multi-user, no authentication, no network-exposed services (except optional Prometheus metrics endpoint on localhost).
- No changes to the core macro-cycle logic, FSM transitions, or gate evaluation.

## 1.3 New Crates

Phase 2 introduces at most 2 new crates:

| Crate                 | Responsibility                                                                                                                                                                                                                           |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>fsr-tui</code>  | TUI dashboard: layout, widgets, input handling, render loop. Depends on <code>ratatui</code> , <code>crossterm</code> . Reads state from <code>fsr-runtime</code> via shared <code>Arc&lt;DashboardState&gt;</code> .                    |
| <code>fsr-ttcp</code> | TTCP v2.0.0 analysis engine: persistent homology computation, cascade levels, convergence tracking, Crystal emission. Depends on <code>fsr-types</code> , <code>fsr-fixed</code> , <code>fsr-resonance</code> , <code>fsr-chain</code> . |

All other Phase 2 features are added as modules within existing crates.

### Crate Budget

The total workspace must not exceed 16 crates after Phase 2. (14 existing + 2 new = 16.)

## 2 TUI Dashboard

### 2.1 Technology

- `ratatui` 0.28+ (terminal rendering framework).
- `crossterm` (cross-platform terminal backend).
- No `tokio` in the render thread. TUI runs on a dedicated OS thread reading from a shared state struct.

### 2.2 Layout

The TUI has a single-screen layout with 6 panels:

Listing 1: TUI layout (approximate)

```

+--[ FIXPOINT SWARM-R v3.0.0 ]--[ tick: 4821 ]--[ paper ]--+
|
| [1] Regime & Gates          [2] Resonance Metrics          |
| Regime: Alpha              kappa: 0.7234                  |
| Gate: OPEN (G=0.712)      H: 0.4521                     |
| Hedge: Safe                sync: 0.6812                  |
| CSP: Idle                  M: 0.3624                     |
| Integrity: Healthy         SI: 1247 bp                   |
| Resource: Nominal          Leak: 0.0312                  |
|                             psi/rho/omega: .81/.74/.69    |
|-----+-----|
| [3] Candidates & Trumpet   | [4] P&L & Execution          |
| L1 tri: 12 candidates      | Net PnL: +47.3 bp          |
| L2 quad: 3 candidates      | Gross: +52.1 bp          |
| L3 hedge: 0                 | Fees: -4.8 bp           |
| L4 EQ: 1                    | Trades: 23               |
| Top-k: 3 after press        | Win rate: 65.2%          |
| Best SI: 1891 bp           | Max DD: 12.4 bp          |
|-----+-----|
| [5] Event Log (scrollable) |                          |
| 04:12:03.441 [CSP] Idle -> Discovered (SI=1891)          |
| 04:12:03.442 [GATE] Regime=pass Mirror=pass Tempo=pass   |
| 04:12:03.443 [CSP] Discovered -> IntentOpen              |
| 04:12:03.501 [CSP] IntentOpen -> Abort (slip exceeded)    |
|-----+-----|
| [6] TTCP Status           |                          |
| Cascade depth: 3/4   d1=0.0412 d2=0.0891 d3=0.1204      |
| Crystals emitted: 0 Last crystal: never                  |
+-----+-----+
| [q]uit [p]ause [r]esume [c]onfig [+/-] speed            |

```

+-----+

## 2.3 Panel Specifications

| # | Panel             | Data Source                                                             | Update Frequency                 |
|---|-------------------|-------------------------------------------------------------------------|----------------------------------|
| 1 | Regime & FSMs     | All 7 FSM current states<br>+ gate score $\Gamma$                       | Every macro-cycle tick           |
| 2 | Resonance Metrics | ResonanceSnapshot<br>from <b>fsr-resonance</b>                          | Every tick                       |
| 3 | Candidates        | Trumpet layer counts,<br>top- $k$ after press, best SI                  | Every tick                       |
| 4 | P&L               | Cumulative from<br>execution receipts                                   | Every settled CSP                |
| 5 | Event Log         | Last 100 events from<br>shadow-chain                                    | Real-time append                 |
| 6 | TTCP Status       | Convergence distances<br>$\delta_1, \delta_2, \delta_3$ , crystal count | Every TTCP cascade<br>completion |

## 2.4 Shared State Architecture

Listing 2: DashboardState struct

```
pub struct DashboardState {
  pub tick: u64,
  pub mode: RunMode,           // Paper | Live | Replay
  pub regime: RegimeState,
  pub csp: CspState,
  pub hedge: HedgeState,
  pub integrity: IntegrityPosture,
  pub resource: ResourcePosture,
  pub promotion: PromotionState,
  pub gate_score: Q32,
  pub gate_open: bool,
  pub resonance: ResonanceSnapshot,
  pub candidates: CandidateSummary,
  pub pnl: PnlSummary,
  pub events: VecDeque<DashboardEvent>, // ring buffer, max 200
  pub ttcp: TtcpSummary,
  pub config_hash: Hash256,
}
```

The runtime writes to `Arc<Mutex<DashboardState>` at the end of each macro-cycle. The TUI thread reads from the same Arc at its own render rate (target: 10 fps). The Mutex is held for microseconds (single struct copy). No contention issues expected.

## 2.5 Keybindings

| Key                   | Action                                                      |
|-----------------------|-------------------------------------------------------------|
| q                     | Graceful shutdown (finish current tick, flush chains, exit) |
| p                     | Pause macro-cycle (observation continues, execution paused) |
| r                     | Resume from pause                                           |
| c                     | Trigger config reload from disk                             |
| + / -                 | Adjust paper-mode tick speed ( $\times 0.5$ , $\times 2$ )  |
| Tab                   | Cycle focus between panels (for scrolling event log)        |
| $\uparrow/\downarrow$ | Scroll focused panel                                        |

## 2.6 TUI Launch

The TUI is activated by default when running `fsr run-paper` or `fsr run-live`. It can be disabled with `-no-tui` for headless/CI operation. All existing CLI commands (`replay`, `audit`, `validate-config`, etc.) remain non-TUI.

### TUI Rule

The TUI is purely read-only visualization. It never modifies system state. If the TUI panics, the engine continues running headless. The TUI must not block the macro-cycle.

## 3 Persistence Layer

### 3.1 What Gets Persisted

| Data                    | Format           | Location                            |
|-------------------------|------------------|-------------------------------------|
| Shadow-chain events     | Bincode (append) | data/shadow/{run_id}/{segment}.bin  |
| Commitment-chain events | Bincode (append) | data/commit/{run_id}/{segment}.bin  |
| Snapshots (periodic)    | Bincode          | data/snapshots/{run_id}/{tick}.snap |
| Configuration           | YAML (copy)      | data/runs/{run_id}/config.yaml      |
| TTCP Crystals           | JSON             | data/crystals/{crystal_id}.json     |
| Run metadata            | JSON             | data/runs/{run_id}/meta.json        |

### 3.2 Chain Persistence

Listing 3: Chain file writer

```
pub struct ChainFileWriter {
  path: PathBuf,
  file: BufWriter<File>,
```

```

    events_in_segment: u64,
    max_events_per_segment: u64, // default: 100_000
    current_segment: u64,
}

impl ChainSink for ChainFileWriter {
    fn append(&mut self, event: ChainEvent) -> Result<Hash256, ChainError> {
        let bytes = bincode::serialize(&event)?;
        self.file.write_all(&(bytes.len() as u32).to_le_bytes())?;
        self.file.write_all(&bytes)?;
        self.events_in_segment += 1;
        if self.events_in_segment >= self.max_events_per_segment {
            self.rotate_segment()?;
        }
        Ok(event.digest)
    }
}

```

### 3.3 Snapshot and Resume

Every  $N$  ticks (configurable, default: 1000), the engine writes a full `SystemSnapshot` to disk. On restart with `fsr run-paper -resume {run_id}`, the engine:

1. Loads the latest snapshot.
2. Replays the shadow-chain from the snapshot's chain head to the chain tail.
3. Resumes the macro-cycle from the restored state.

#### Persistence Rule

The engine must be stoppable and resumable without data loss. A crash at any point must not corrupt the chain files (write-ahead: event is flushed before the engine proceeds to the next tick). Partial writes are detected by length-prefix validation on reload.

### 3.4 Compaction

Old shadow-chain segments are compressed with `zstd` after they are  $> 24$ h old. The compaction runs in a background task and never blocks the macro-cycle.

## 4 Live Venue Adapter: Binance Spot

### 4.1 Scope

One adapter: Binance Spot. Read-only by default (order book observation). Execution requires `-features live-exec` and API key configuration.

### 4.2 Implementation



Listing 4: Binance adapter (feature-gated)

```
// In fsr-runtime, behind #[cfg(feature = "live")]
pub struct BinanceAdapter {
    rest_client: request::Client,
    ws_handle: Option<tokio::task::JoinHandle<()>>,
    orderbook_cache: Arc<Mutex<HashMap<TradingPair, OrderBook>>>,
    api_key: Option<String>,    // None = read-only
    api_secret: Option<String>,
}

impl VenueAdapter for BinanceAdapter {
    fn venue_id(&self) -> VenueId { VenueId::Binance }

    async fn fetch_orderbook(&self, pair: &TradingPair)
        -> Result<OrderBook, VenueError>
    {
        // Return from WebSocket cache (updated in background)
        // Fallback to REST if WS disconnected
    }

    async fn submit_order(&self, order: &OrderRequest)
        -> Result<OrderReceipt, VenueError>
    {
        #[cfg(not(feature = "live-exec"))]
        return Err(VenueError::ExecutionDisabled);

        #[cfg(feature = "live-exec")]
        // POST to Binance API with HMAC signature
    }
}
```

### 4.3 WebSocket Order Book Feed

1. Connect to `wss://stream.binance.com:9443/ws/{symbol}@depth20@100ms`.
2. Parse depth snapshots into canonical `OrderBook` struct.
3. Convert all prices/quantities from floating-point strings to Q32 at the **adapter boundary** (before entering the decision pipeline).
4. Store in `orderbook_cache`. The macro-cycle reads from cache at each tick.
5. On disconnect: retry with exponential backoff (1s, 2s, 4s, max 30s). Log as `ExternalDependencyFailure` if disconnected > 60s.

### 4.4 Configuration

Listing 5: Binance config section in YAML

```
venues:
  binance:
    enabled: true
    pairs: ["BTCUSDT", "ETHUSDT", "ETHBTC"]
    ws_depth_level: 20          # 5, 10, or 20
```

```
ws_update_ms: 100          # 100ms or 1000ms
rest_fallback: true
api_key_env: "BINANCE_API_KEY"      # read from env var
api_secret_env: "BINANCE_API_SECRET"
execution_enabled: false # must be true + feature flag
```

### Live Safety Rules

1. API keys are **never** stored in config files. Read from environment variables only.
2. Execution is double-gated: compile-time (`-features live-exec`) AND runtime (`execution_enabled: true`).
3. The first 100 ticks after live-mode start are forced paper-mode (observation only) regardless of config. This gives the system time to calibrate.
4. All live orders require a valid NullcenterCert (INV-01 enforced).
5. Live-mode emits a `LiveModeActivated` event to the commitment chain on start.

## 5 Observability

### 5.1 Structured Logging

Replace all `println!` and `eprintln!` with tracing spans and events:

Listing 6: Tracing integration

```
// At macro-cycle level:
let span = tracing::info_span!("macro_cycle", tick = tick, regime = ?regime);
let _guard = span.enter();

tracing::info!(si = %si, gate_open = gate_open, "gate evaluated");
tracing::warn!(failure = ?failure, "invariant check failed");
```

Log output formats (configurable):

- `json`: structured JSON lines to `data/logs/{run_id}.jsonl`.
- `pretty`: human-readable to `stderr` (default in TUI mode: disabled, logs go to file only).
- `compact`: one-line per event to `stderr` (default in headless mode).

### 5.2 Metrics Export

Optional Prometheus-compatible metrics endpoint on `localhost:9090/metrics` (gated behind `-metrics`):

Listing 7: Key metrics

|                                          |         |                                         |
|------------------------------------------|---------|-----------------------------------------|
| <code>fsr_tick_total</code>              | counter | Total macro-cycle ticks                 |
| <code>fsr_regime_state</code><br>(Gamma) | gauge   | Current regime (0=Alpha,1=Beta,2=Gamma) |
| <code>fsr_gate_score</code>              | gauge   | Current Gamma(x) value                  |
| <code>fsr_si_current</code>              | gauge   | Current Spiral Index (bp)               |

|                               |         |                              |
|-------------------------------|---------|------------------------------|
| fsr_pnl_net_bp                | gauge   | Net P&L in basis points      |
| fsr_csp_settled_total         | counter | Total settled CSP cycles     |
| fsr_csp_aborted_total         | counter | Total aborted CSP cycles     |
| fsr_hedge_triggered_total     | counter | Total hedge triggers         |
| fsr_ttcp_delta1               | gauge   | TTCP L1 convergence distance |
| fsr_ttcp_delta2               | gauge   | TTCP L2 convergence distance |
| fsr_ttcp_delta3               | gauge   | TTCP L3 convergence distance |
| fsr_ttcp_crystals_total       | counter | Total TTCP crystals emitted  |
| fsr_chain_shadow_events_total | counter | Shadow-chain event count     |
| fsr_chain_commit_events_total | counter | Commitment-chain event count |

## 6 TTCP Analysis Engine

### 6.1 Integration Point

The TTCP engine runs as a background task alongside the macro-cycle. At each macro-cycle tick, the engine receives the current `ResonanceSnapshot` ( $\kappa, H, \text{sync}, M, \text{SI}, \text{Leak}, \psi, \rho, \omega$ ) and appends it to its observation window.

### 6.2 Embedding

Each `ResonanceSnapshot` is embedded into  $H_5$  via:

$$e(t) = (\psi(t), \rho(t), \omega(t), \text{SI}(t)/\text{SI}_{\max}, \text{Leak}(t))$$

All values are already Q32 from `fsr-resonance`. The embedding is a direct read, no conversion needed.

### 6.3 Simplified TTCP for Phase 2

The full TTCP v2.0.0 with Vietoris-Rips persistent homology is computationally expensive. For Phase 2, implement a **simplified but correct** version:

1. **Level 1 (5D)**: Instead of full VR filtration, use the Betti-0 approximation: count connected components at reference scale  $\varepsilon^*$  via single-linkage clustering. This gives  $\beta_0$  (number of clusters). Track  $\beta_0$  over time.
2. **Level 2 (4D fibers)**: Partition the observation window by  $\omega$  (frequency axis) into  $M_f$  bins. For each bin, compute  $\beta_0$  via the same clustering. Track the Betti trajectory  $\mathcal{B} = (\beta_0(0), \dots, \beta_0(M_f - 1))$ .
3. **Level 3 (meta)**: Treat the Betti trajectory  $\mathcal{B}$  as a point in  $\mathbb{R}^{M_f}$ . Track  $\|\mathcal{B}^{(w)} - \mathcal{B}^{(w-1)}\|_1$  as the meta-convergence distance.
4. **Convergence**: All three distances monotonically decreasing over a window of size  $s_{\min}$  AND all below  $\Theta$ .
5. **Crystal**: Emit a Crystal artifact (JSON) with Betti signatures, convergence distances, and evidence digest.

### TTCP Phase 2 vs. Full TTCP

This simplified version uses  $\beta_0$  only (connected components) instead of full persistent homology  $(\beta_0, \beta_1, \dots, \beta_4)$ . It captures the *structure* of the TTCP protocol — three-level cascade, convergence tracking, Crystal emission — without the  $O(n^5)$  VR filtration cost. The full persistent homology engine is Phase 3 scope (when a Rust PH library is integrated).

## 6.4 TTCP Configuration

Listing 8: TTCP config section

```
ttcp:
  enabled: true
  window_size: 200          # observations per window
  window_stride: 50         # overlap
  reference_epsilon: 500    # Q32-scaled clustering threshold
  fiber_axis: 3             # omega (index 0-4 in embedding)
  fiber_count: 8            # M_f
  condensation_window: 5    # s_min
  crystallization_threshold: 100 # Theta (Q32-scaled)
  crystal_output_dir: "data/crystals"
```

## 7 Configuration Hot-Reload

### 7.1 Mechanism

1. Operator presses `c` in TUI (or sends `SIGHUP` in headless mode).
2. Engine reads YAML from disk.
3. Engine validates new config against schema (same as `fsr validate-config`).
4. If valid: compute diff. Log changes. Swap config atomically. Emit `ConfigReloaded` event with old/new `CfgHash`.
5. If invalid: log error. Keep old config. Display error in TUI.

### 7.2 What Can Be Changed at Runtime

- Gate thresholds ( $\theta_{\text{open}}, \theta_{\text{close}}$ , gate weights).
- Risk parameters (leverage cap, risk budget, `dd_max`).
- Trumpet layer config (candidate counts, top- $k$ ).
- Calibration parameters.
- TTCP parameters.
- Logging verbosity.

### 7.3 What Cannot Be Changed at Runtime

- Venue list (requires restart).
- Trading pairs (requires restart).

- Q32 arithmetic settings.
- Chain genesis parameters.

### Hot-Reload Rule

Every config change must be NC-factored: the `ConfigReloaded` event carries old and new `CfgHash` and is appended to the shadow-chain. This makes all parameter changes replay-visible.

## 8 Implementation Order

| Step | Component         | Done when                                                                                                                                                                 |
|------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.1  | Persistence       | Chain files write/read/rotate. Snapshot save/load. <code>fsr run-paper</code> writes to <code>data/</code> . <code>-resume</code> works.                                  |
| 2.2  | Observability     | <code>tracing</code> integrated in all crates. JSON log output. <code>-metrics</code> endpoint serves counters/gauges.                                                    |
| 2.3  | TUI core          | <code>fsr-tui</code> crate. Dashboard renders all 6 panels with mock data. Keybindings work.                                                                              |
| 2.4  | TUI integration   | <code>fsr run-paper -tui</code> (default) shows live dashboard fed from actual macro-cycle. <code>-no-tui</code> falls back to headless.                                  |
| 2.5  | Config hot-reload | <code>c</code> key triggers reload. Diff logged. Invalid config rejected. Event emitted.                                                                                  |
| 2.6  | Binance adapter   | <code>fsr-runtime</code> gains <code>BinanceAdapter</code> . WebSocket feed. Q32 conversion. <code>fsr run-paper -venue binance</code> observes real books in paper-mode. |
| 2.7  | TTCP engine       | <code>fsr-ttcp</code> crate. Simplified 3-level cascade. Crystal emission. Panel 6 in TUI shows TTCP status.                                                              |
| 2.8  | Integration test  | End-to-end: <code>fsr run-paper -config balanced.yaml</code> runs 1000 ticks with TUI, persistence, TTCP, and observability. All Phase 1 tests still pass.                |

### Step Closure Rule

Each step must compile, pass tests, and not break prior steps before the next begins. Steps 2.1–2.2 are infrastructure. Steps 2.3–2.5 are the TUI layer. Step 2.6 is the live adapter. Step 2.7 is the TTCP engine. Step 2.8 is the integration gate.

## 9 New Dependencies

| Crate              | Version     | Purpose                                      |
|--------------------|-------------|----------------------------------------------|
| ratatui            | $\geq 0.28$ | TUI rendering                                |
| crossterm          | $\geq 0.28$ | Terminal backend                             |
| tracing            | $\geq 0.1$  | Structured logging                           |
| tracing-subscriber | $\geq 0.3$  | Log formatting (JSON, pretty)                |
| request            | $\geq 0.12$ | HTTP client (Binance REST, feature-gated)    |
| tokio-tungstenite  | $\geq 0.24$ | WebSocket client (Binance WS, feature-gated) |
| bincode            | $\geq 2.0$  | Binary serialization for chain files         |
| zstd               | $\geq 0.13$ | Compression for old chain segments           |
| prometheus-client  | $\geq 0.22$ | Metrics export (optional)                    |

### Dependency Rule

All network-dependent crates (`request`, `tokio-tungstenite`) must be behind feature flags. `cargo build -workspace` without features must succeed using only local/synthetic data.

## 10 Updated CLI Surface

Phase 2 adds 2 new commands and modifies 1:

Listing 9: Updated CLI

```
# Modified: now launches TUI by default
fsr run-paper --config <path> [--no-tui] [--resume <run_id>]

# New: live observation mode (paper-mode with real data)
fsr run-live --config <path> --venue binance [--no-tui]

# New: list past runs and their status
fsr runs [--last <n>]
```

All existing commands (`replay`, `audit`, `validate-config`, `validate-promotion`, `rollback-check`, `resource-posture`, `status`) remain unchanged.

## 11 File System Layout After Phase 2

Listing 10: Extended repository layout

```

fixpoint-swarm-r/
  crates/
    fsr-types/      # unchanged
    fsr-fixed/      # unchanged
    fsr-temporal/   # unchanged
    fsr-nullcenter/ # unchanged
    fsr-resonance/  # unchanged
    fsr-gate/       # unchanged
    fsr-mirror/     # unchanged
    fsr-candidates/ # unchanged
    fsr-csp/        # unchanged
    fsr-hedge/      # unchanged
    fsr-calibration/ # unchanged
    fsr-chain/      # + file-backed ChainSink impl
    fsr-governance/ # unchanged
    fsr-runtime/    # + persistence, Binance adapter, hot-reload
    fsr-tui/        # NEW: TUI dashboard
    fsr-ttcp/       # NEW: TTCP analysis engine
  config/          # unchanged
  data/            # NEW: runtime data directory
    shadow/        # chain segments
    commit/        # commitment chain
    snapshots/     # periodic state snapshots
    crystals/      # TTCP crystal artifacts
    logs/          # structured log files
    runs/          # per-run metadata
  tests/
    integration/   # + Phase 2 integration tests
  ci/
    run-all.sh     # + Phase 2 test steps

```

## 12 Definition of Done (Phase 2)

Phase 2 is complete when:

1. All Phase 1 tests pass unchanged (`cargo test -workspace`).
2. `cargo clippy -workspace - -D warnings` passes.
3. `fsr run-paper -config balanced.yaml` runs 1000 ticks with TUI, writes chain files, writes snapshots, and exits cleanly on `q`.
4. `fsr run-paper -resume {run_id}` resumes from last snapshot and continues correctly.
5. `fsr run-paper -no-tui` runs headless with JSON log output.
6. `fsr run-live -config balanced.yaml -venue binance` connects to Binance WebSocket, displays real order book data in TUI, runs macro-cycle in paper-mode with real data.
7. TTCP panel shows convergence distances updating in real-time.
8. Config hot-reload via `c` key works and emits evidence event.

9. `fsr runs` lists past runs with tick count and status.
10. `-metrics` endpoint serves Prometheus-compatible metrics on localhost.
11. `data/` directory structure is created automatically on first run.
12. No network access required for `fsr run-paper` (fully offline).

## 13 Phase 3 Preview (Non-Normative)

---

For planning purposes only. Phase 3 will likely include:

- Additional venue adapters (Kraken, Uniswap).
- Full persistent homology (Ripser/GUDHI bindings) for TTCP.
- Promotion workflow: paper-validated configs promoted to live via CLI.
- Historical data replay from recorded order book files.
- Performance profiling and optimization of hot paths.

---

End of Phase 2 Specification